

APIs

APIs that interact with our database

Type	End Point	Description
Authentication & users		
POST	/api/auth/register	Register a new user — stores user record, hashed password, default settings
POST	/api/auth/login	Authenticate user — returns JWT access token + refresh token
POST	/api/auth/refresh-token	Issue new access token using valid refresh token
POST	/api/auth/logout	Invalidate refresh token — revoke session from DB
GET	/api/auth/me	Fetch authenticated user's profile and active settings
PUT	/api/auth/me	Update user profile (name, email, preferences)
POST	/api/auth/change-password	Change password with old-password verification
Broker credentials		
POST	/api/broker/credentials	Save Zerodha API key and secret (encrypted at rest in DB)
GET	/api/broker/credentials	Fetch saved broker credential record (masked key) for the user
PUT	/api/broker/credentials	Update Zerodha API key / secret
DELETE	/api/broker/credentials	Remove stored broker credentials
GET	/api/broker/session-status	Check if Zerodha access token is active and not expired (reads DB state)
Sector & stock master		
GET	/api/master/sectors	List all sectors from sector_master (active + inactive)
POST	/api/master/sectors	Create new sector entry in sector_master
PUT	/api/master/sectors/:sector_id	Update sector name or active status
GET	/api/master/stocks	List all stocks from stock_master with sector mapping
POST	/api/master/stocks	Add stock to stock_master with instrument_key and sector_id
PUT	/api/master/stocks/:stock_code	Update stock mapping, sector assignment, or is_active flag

POST	/api/master/stocks/bulk-import	Bulk import stocks from CSV/JSON into stock_master
Watchlist management		
GET	/api/watchlist	Fetch all watchlist_upload entries for the user
POST	/api/watchlist/upload	Upload watchlist (CSV/JSON) — triggers 20-day historical fetch job
POST	/api/watchlist/add	Add single stock to watchlist with entry_price, stop_loss, target, target_mode
PUT	/api/watchlist/:stock_code	Update entry_price, SL, target, or target_mode for a tracked stock
DELETE	/api/watchlist/:stock_code	Remove stock from watchlist (only if not in ACTIVE or ORDER_PLACED state)
Historical & market data (stored)		
GET	/api/data/historical/:stock_code	Fetch last 20 daily candles from historical_daily_data table
POST	/api/data/historical/refresh/:stock_code	Trigger manual re-fetch of 20-day candles for a stock
GET	/api/data/intraday-candles/:stock_code	Fetch stored 15-min intraday candles from intraday_candles table for a given date
GET	/api/data/volume-baseline/:stock_code	Read avg_volume_10d / avg_volume_20d per time-slot from volume_baseline table
GET	/api/data/live-signals	Fetch latest live_volume_signal rows for all tracked stocks
Metrics & dashboard reads		
GET	/api/metrics/market	Read latest market_metrics row — Nifty change, market_status
GET	/api/metrics/sectors	Read sector_metrics for all sectors — ranked with status (Sector Dashboard data)
GET	/api/metrics/stocks	Read stock_metrics for all tracked stocks — price_change_pct, volume_ratio, status
GET	/api/metrics/stocks/:stock_code	Read stock_metrics for a single stock
GET	/api/tracked-stocks	Tracked Stock Screen data — returns entry_status, volume_ratio, sector_status, confirmation_status per stock
Strategy configuration		
GET	/api/strategies	List all strategy configs from DB (Model 1, Model 2, future dynamic ones)

GET	/api/strategies/:strategy_id	Fetch full strategy config JSON including version info
POST	/api/strategies	Create new strategy config — runs Zod schema validation before save
PUT	/api/strategies/:strategy_id	Update strategy (bumps version, old trades continue on old version)
POST	/api/strategies/:strategy_id/activate	Mark strategy as active for new trades (deactivates previous active version)
GET	/api/strategies/:strategy_id/versions	List all versions of a strategy for version history view

Trades, orders & positions

GET	/api/trades	List all trade_positions (active + closed) with pagination
GET	/api/trades/active	Fetch only ACTIVE and TRAILING positions — Invested Stock Screen data
GET	/api/trades/:trade_id	Full detail of a single trade including entry, SL, targets, pnl_pct
GET	/api/trades/:trade_id/orders	All trade_orders rows for a trade — shows broker order sequence
GET	/api/trades/:trade_id/logs	All trade_logs entries for a trade — audit trail of every action
POST	/api/trades/:trade_id/manual-exit	Trigger manual exit — sets EXIT_TRIGGERED in state machine, queues sell order
GET	/api/orders	Paginated list of all trade_orders rows across all trades

Risk settings & configuration

GET	/api/settings/risk	Fetch risk config — capital_per_trade, max_live_trades, weak_market_qty_pct
PUT	/api/settings/risk	Update risk parameters (only applies to new trades, not running ones)
GET	/api/settings/system	Fetch system config — engine refresh intervals, scheduler toggles
PUT	/api/settings/system	Update system-level config (engine on/off, cooldown periods)

Analytics, alerts & logs

GET	/api/analytics/performance	Aggregate performance stats — win rate, avg pnl, max drawdown from closed trades
GET	/api/analytics/trade-history	Paginated closed trade history with filters (date range, stock, strategy)

GET	/api/alerts	Fetch all unread alerts from alerts table (failures, SL hits, signals)
PATCH	/api/alerts/:alert_id/seen	Mark a single alert as seen (seen_flag = true)
POST	/api/alerts/mark-all-seen	Mark all unread alerts as seen in bulk
GET	/api/logs/system	Fetch structured system-level logs (Pino output stored in DB or file), paginated

Trade state & duplicate control

GET	/api/trades/duplicate-check/:stock_code	Check if stock already has ACTIVE, TRAILING, or ORDER_PLACED position — returns true/false before placing any order
GET	/api/trades/cooldown/:stock_code	Check if stock is in 1-trading-day cooldown after closure — returns cooldown_active flag and expires_at timestamp
POST	/api/trades/:trade_id/force-state	Admin override to manually set trade state in state machine — used for crash recovery or stuck states only

Maintenance & engine control

POST	/api/maintenance/end-of-day	Manually trigger end-of-day job — refreshes avg_10d_volume, stores daily candles, resets daily metrics
POST	/api/engine/pause	Pause all engines (sector, tracked, invested) — stops new signals and order execution
POST	/api/engine/resume	Resume all paused engines — re-enables signal generation and order execution
GET	/api/engine/status	Fetch current running/paused status of each engine with last_run timestamp and next_run schedule

Strategy validation

POST	/api/strategies/:strategy_id/validate	Dry-run Zod schema validation on a strategy config without saving — returns pass/fail with error details per field
-------------	---------------------------------------	--

APIs that interact with Zerodha Kite

Type	End Point	Description
Authentication & session		

GET	/api/broker/kite/login-url	Generate Zerodha Kite login URL — opens OAuth flow for user to authorize
POST	/api/broker/kite/callback	Receive request_token from Kite redirect → call Kite generateSession() → store access_token in DB
DELETE	/api/broker/kite/session	Invalidate Kite access token — calls Kite invalidateSession() + clears DB token
GET	/api/broker/kite/profile	Fetch Kite user profile (name, email, segments) — calls Kite getProfile()
GET	/api/broker/kite/margins	Fetch available capital and margins per segment from Kite — calls Kite getMargins()

Instruments & market data

GET	/api/broker/kite/instruments	Download full NSE instruments dump from Kite — used to populate stock_master instrument_keys
GET	/api/broker/kite/quote:instruments	Fetch full market quote (LTP, OHLC, volume, prev_close) for up to 500 instruments
GET	/api/broker/kite/ltip:instruments	Fetch LTP only for given instruments (lighter call than full quote)
GET	/api/broker/kite/ohlc:instruments	Fetch OHLC + LTP for given instruments (used for intraday checks)
GET	/api/broker/kite/historical:instrument_token	Fetch historical candles — query params: from, to, interval (day / 15minute). Stores results in historical_daily_data or intraday_candles

Order placement & management

POST	/api/broker/kite/orders/place	Place new buy/sell order on Kite — sends symbol, qty, order_type, price, trigger_price
PUT	/api/broker/kite/orders/:order_id	Modify existing order on Kite — update price, qty, or trigger_price (for SL modification)
DELETE	/api/broker/kite/orders/:order_id	Cancel pending order on Kite
GET	/api/broker/kite/orders	Fetch all orders for current trading day from Kite — used for reconciliation
GET	/api/broker/kite/orders/:order_id	Fetch status of a specific order from Kite — COMPLETE / REJECTED / CANCELLED / OPEN
GET	/api/broker/kite/orders/:order_id/trades	Fetch fills/trades for a specific order — used to get actual filled_qty and filled_price

Positions, holdings & GTT

GET	/api/broker/kite/positions	Fetch net and day positions from Kite — used for reconciliation with trade_positions table
------------	----------------------------	--

GET	/api/broker/kite/holdings	Fetch long-term holdings from Kite demat account
POST	/api/broker/kite/gtt/place	Place GTT (Good Till Triggered) stop-loss order on Kite for overnight SL protection
PUT	/api/broker/kite/gtt/:gtt_id	Modify GTT trigger price (when trailing SL moves up)
DELETE	/api/broker/kite/gtt/:gtt_id	Delete GTT order (on trade close or manual cancellation)
GET	/api/broker/kite/gtt	List all active GTT orders from Kite for reconciliation

Order history & margin checks

GET	/api/broker/kite/orders/history	Fetch Kite order history for past N trading days — used for reconciliation and trade audit beyond today
POST	/api/broker/kite/margin/calculate	Call Kite margin calculator API before placing order — verifies sufficient capital is available for the trade
POST	/api/broker/kite/orders/basket	Place multiple orders in one call (basket order) — used for simultaneous partial exit legs in Model 2 trailing strategy
POST	/api/broker/kite/orders/sl-verify	Verify SL trigger price is valid before placement — checks tick size, circuit limits, and price band from Kite

WebSocket connections

Type	End Point	Description
Our WebSocket server → clients (frontend)		
WS	wss://ourserver/ws	Main WebSocket connection endpoint — frontend connects here with JWT in handshake header

Events we emit to frontend

EMIT	market:status	Broadcast market_status update — STRONG / NEUTRAL / WEAK — on every market engine refresh
EMIT	sector:metrics:update	Push updated sector_metrics object (all sectors ranked) every 5 minutes
EMIT	stock:ltp	Stream live LTP, volume_ratio, price_change_pct per tracked stock — every tick forwarded from Kite
EMIT	stock:entry_status	Push entry_status change — WAITING / BLOCKED / READY — for Tracked Stock Screen live updates
EMIT	stock:confirmation_timer	Push 5-minute confirmation countdown tick per stock entering WAITING_CONFIRMATION state

EMIT	trade:signal	Notify frontend when BUY signal is generated — includes stock_code, entry_price, strategy
EMIT	trade:order_update	Stream order state changes — ORDER_PLACED → FILLED / PARTIAL / REJECTED — in real time
EMIT	trade:position_update	Push live position updates — pnl_pct, current_price, stop_loss, position_status every 30s
EMIT	trade:exit_triggered	Notify frontend when any exit rule fires — includes exit_reason and stock_code
EMIT	trade:closed	Notify frontend when trade reaches CLOSED state with final pnl_pct and exit_reason
EMIT	alert:new	Push new alert to frontend in real time (SL failure, order rejection, API error)
EMIT	volume:signal	Push 15-minute volume signal result — slot_ratio, cumulative_ratio, signal_status per stock
EMIT	system:engine_heartbeat	Periodic heartbeat from each engine (sector, tracked, invested) — shows last_run and status
EMIT	system:error	Push critical system error to frontend — API failure, DB timeout, broker disconnect

Events our server listens for from frontend

ON	subscribe:stocks	Frontend sends list of stock_codes to subscribe for live LTP and signal updates
ON	unsubscribe:stocks	Frontend sends stock_codes to unsubscribe from live streaming
ON	alerts:mark_seen	Frontend sends alert_id to mark seen via WebSocket (alternative to REST PATCH)
ON	ping	Keepalive from frontend — server responds with pong to maintain connection

Zerodha Kite Ticker WebSocket (Kite → our server)

WS	wss://ws.kite.trade	Our backend connects to Kite WebSocket Ticker using access_token for live market data streaming
-----------	---------------------	---

Kite ticker events our server handles

ON	ticker.connect	Kite connection established — triggers instrument subscription for all tracked + invested stocks
-----------	----------------	--

ON	ticker.ticks	Receive tick array — each tick has instrument_token, LTP, volume, OI, bid/ask. Processed by market data engine
ON	ticker.disconnect	Kite disconnected — our server triggers reconnect logic with exponential backoff, logs alert
ON	ticker.error	Kite ticker error — logged, alert created, reconnect attempted
ON	ticker.reconnect	Kite auto-reconnect attempt event — tracked for monitoring dashboard
ON	ticker.noreconnect	Kite gives up reconnecting — triggers high-priority system alert, pauses all engines
ON	ticker.order_update	Receive real-time order status postbacks from Kite — COMPLETE / REJECTED / CANCELLED — feeds order state machine directly

Commands our server sends to Kite Ticker

EMIT	ticker.subscribe	Subscribe to instrument tokens — sent after connect and when new stocks enter watchlist or positions
EMIT	ticker.unsubscribe	Unsubscribe instrument tokens when stock exits all positions and watchlist
EMIT	ticker.setMode	Set tick mode per instrument — MODE_LTP for tracked stocks, MODE_FULL for invested stocks (includes depth + OI)

Events our server emits to frontend

EMIT	sector:status_change	Push event when a sector flips status mid-session (e.g. STRONG → WEAK) — triggers UI warning and re-evaluation of pending signals
EMIT	trade:sl_updated	Push real-time event when trailing SL moves up — includes stock_code, old_sl, new_sl, trigger reason (5% hit / step_percent)
EMIT	stock:cooldown_status	Push event when a stock enters or exits 1-trading-day cooldown — frontend disables re-entry UI accordingly
EMIT	stock:confirmation_cancelled	Push event when 5-min confirmation timer is cancelled because price dropped below entry during the hold window
EMIT	engine:paused	Push event when market goes WEAK and engines stop processing new entries — includes reason and affected engine list
EMIT	engine:resumed	Push event when engines resume after being paused — frontend re-enables signal display

Kite Ticker event our server handles

ON

ticker.message

Handle raw Kite WebSocket frames including internal Kite ping/pong keepalive — prevents connection timeout during low-activity periods
